BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Trương Hoàng Đăng Khôi
MSSV / Lớp	3124720035 - DNT1241

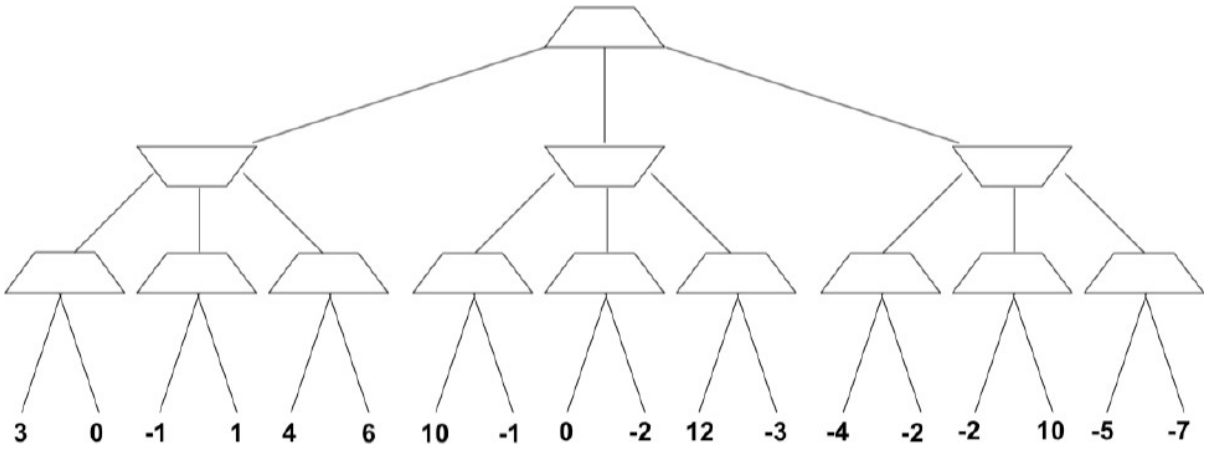
Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Tầng 2 - nút con của các nút MIN:
A1 = max(3, 0) = 3; A2 = max(-1, 1) = 1; A3 = max(4, 6) = 6
B1 = max(10, -1) = 10; B2 = max(0, -2) = 0; B3 = max(12, -3) = 12
C1 = max(-4, -2) = -2; C2 = max(-2, 10) = 10; C3 = max(-5, -7) = -5
Tầng 1 - nút con của Root:
A = min(A1, A2, A3) = min(3, 1, 6) = 1
B = min(B1, B2, B3) = min(10, 0, 12) = 0
C = min(C1, C2, C3) = min(-2, 10, -5) = -5
giá trị nút gốc MAX:
R = max(A, B, C) = max(1, 0, -5) = 1

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

MAX nên chọn nhánh thứ nhất (nhánh A - bên trái cùng) từ nút gốc
Giá trị trò chơi (Game value): 1.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

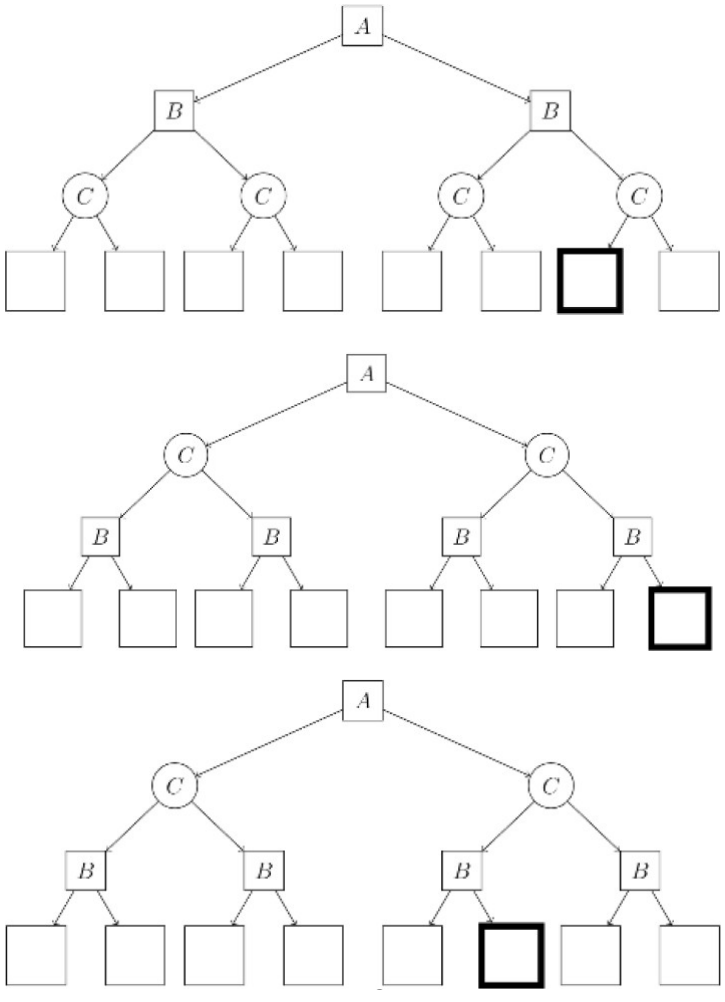
Tại nút A3: Cắt nhánh lá 6 (alpha = 4 >= beta = 1).
Tại nút B: Cắt toàn bộ nhánh B3 gồm 2 lá 12 và -3 (beta = 0 <= alpha = 1).
Tại nút C: Cắt toàn bộ 2 nhánh C2 và C3 gồm 4 lá -2, 10, -5, -7 (beta = -2 <= alpha = 1).

d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

7 nút lá không cần thăm: 6, 12, -3, -2 (của C2), 10, -5, -7.
So sánh số nút lá được đánh giá:
Minimax: 18 nút lá.
Alpha-Beta Pruning: 11 nút lá (tiết kiệm được 7 nút).

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Gán giá trị lá (từ trái sang phải): 2, 2, 2, 2, 2, 2, X, bất kỳ

Điều kiện cắt tỉa:

- Nhánh B bên trái chọn giá trị 2 => Nút A (MIN) cập nhật beta = 2.
- Nhánh B bên phải: Nút C thứ ba thăm lá đầu tiên là 3 => Giá trị nút C = (3 + lá thứ 2)/2 >= 1,5.
- Nếu lá thứ 3 có giá trị là 5 => C = (5+lá 4)/2 >= 2.5.
- Khi đó nút B bên phải (MAX) có giá trị >= 2.5 > beta = 2 => Cắt tỉa lá thứ 7 .

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Kết quả: Không thể vì ô viền đậm là nút lá cuối cùng của cây. Việc tính giá trị nút ngẫu nhiên C (bên phải) yêu cầu phải biết đầy đủ giá trị của cả 2 con để tính trung bình cộng. Do đó bắt buộc phải thăm nút lá này.

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Gán giá trị lá (từ trái sang phải): 1, 1, 1, 1, 0, X, bất kỳ, bất kỳ.
Điều kiện cắt tỉa:

- Nhánh C bên trái có giá trị = 1 => Nút A (MIN) nhận giá trị =< 1 => beta = 1.
- Nhánh C bên phải: Nút B thứ ba thăm lá đầu tiên = 0.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

- Trong cây Minimax, giá trị của nút MAX/MIN chỉ phụ thuộc vào giá trị cực đại/cực tiểu của các con, nên chỉ cần 1 nhánh con vượt ngưỡng alpha/beta là có thể cắt tỉa ngay.
- Trong cây Expectiminimax, giá trị nút C là trung bình cộng của tất cả các con. Giá trị của một nhánh bị ẩn có thể kéo trung bình cộng lên hoặc xuống, trừ khi ta biết chắc chặn trên/chặn dưới của các giá trị lá.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \le V_E$. ☒ Đúng ☐ Sai

Giải thích: Nút MIN luôn chọn giá trị nhỏ nhất. Khi thay bằng nút ngẫu nhiên, trung bình trọng số của các con luôn lớn hơn hoặc bằng giá trị nhỏ nhất đó.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☒ Đúng ☐ Sai

Giải thích: Minimax đảm bảo lợi ích ít nhất là V_M trong trường hợp đối thủ chơi tối ưu tuyệt đối. Khi đối thủ chơi ngẫu nhiên, lợi ích kỳ vọng thu được chỉ có thể bằng hoặc cao hơn V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất

là V_E. ☐ Đúng ☐ Sai

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

Giải thích: V_E đạt được khi dùng Expectimax tối ưu. Minimax chọn hành động phòng thủ nên lợi ích kỳ vọng thu được chỉ đảm bảo >= V_M, có thể nhỏ hơn V_E

☐ Minimax với tiện ích trạng thái cuối +1/−1

☒ Expectimax với tiện ích trạng thái cuối +1/−1

☒ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

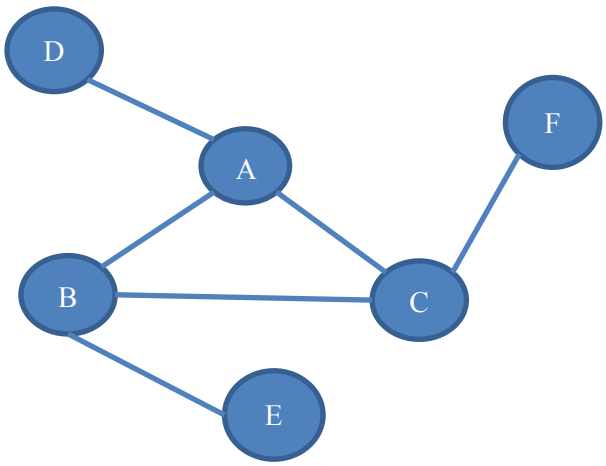
d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

- Expectimax: Phù hợp vì trò chơi có 1 người chơi (MAX) kết hợp với các nút ngẫu nhiên. Giả định: Cây trò chơi hữu hạn và không có vòng lặp.
- Value Iteration: Phù hợp vì đây là bài toán Quy hoạch động Markov với không gian trạng thái và hành động. Giả định: Trò chơi có thể chứa vòng lặp hoặc không gian trạng thái tổng quát hơn.
- Không chọn Minimax: Vì Minimax dành cho trò chơi 2 người chơi đối kháng, không áp dụng cho hành động ngẫu nhiên của môi trường.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Các biến có thể bị thay đổi miền giá trị: B, C, D

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

Các biến có thể bị thay đổi miền giá trị: C, E.

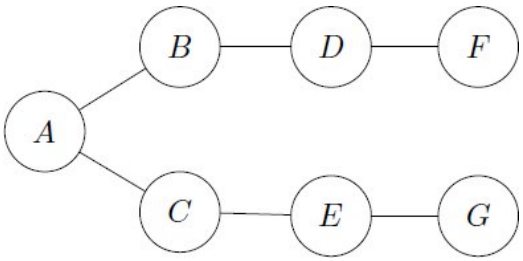
d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

Các biến có thể bị thay đổi miền giá trị: B, C, D, E, F

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

Các biến có thể bị thay đổi miền giá trị: C, D, E, F.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G

Gán xong 2 biến (A, B), AC được kích hoạt. AC sẽ lan truyền đến tất cả các biến còn lại C, D, E, F, G, loại bỏ các giá trị không tương thích.

Số lần backtrack tối đa: 0

g) Thứ tự G → E → C → A → B → D → F

Sau 2 biến (G, E): AC được kích hoạt. AC lan truyền ngược từ E => C => A => B, loại bỏ các giá trị không hợp lệ trên toàn cây.

Sau 4 biến (G, E, C, A): AC tiếp tục chạy sau khi gán A, lan truyền sang nhánh còn lại B, D, F.

Số lần backtrack tối đa: 0 hoặc O(d)

h) Thứ tự G → F → E → D → C → B → A

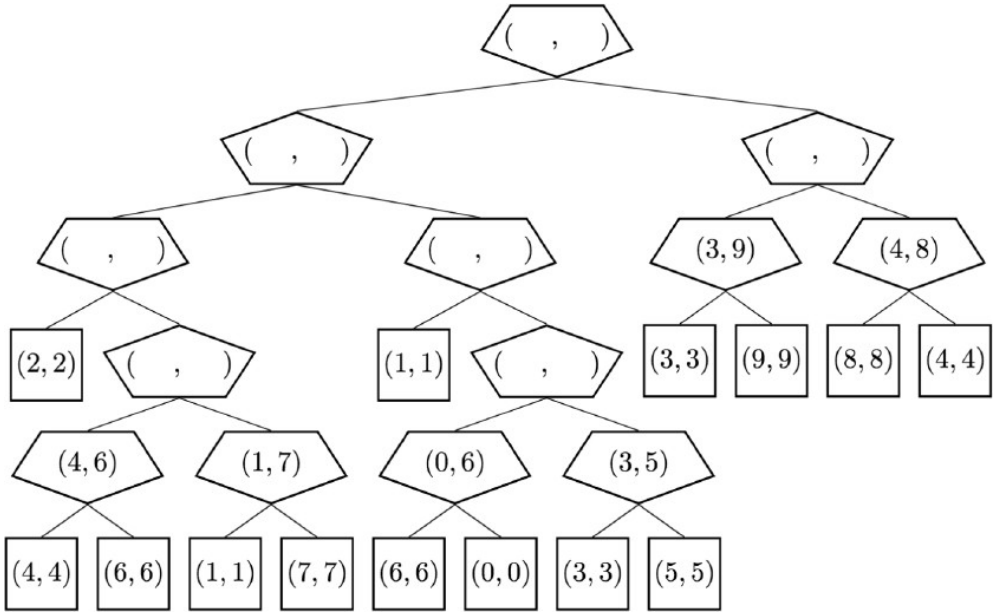
Sau 2 biến (G, F): AC được kích hoạt. Vì G và F thuộc hai nhánh độc lập, AC không thể lan truyền ràng buộc tới các nút chưa gán khác.

Sau 4 biến (G, F, E, D): AC kích hoạt nhưng vẫn chưa kết nối qua A, do đó không phát hiện được mâu thuẫngiữa hai nhánh cho đến khi gán đến A.

Số lần backtrack tối đa: $O(d^6)$ hoặc $O(d^7)$

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

Tất cả cặp (u,v) còn thiếu:

Tầng 1: ô bên trái (6,6), ô bên phải (6,6)

Tầng 2: nút trái (2,7), nút giữa (0,6)

Tầng 3: nút trái (2,7), nút phải (4,8)

Nút gốc: (4, 8)

Sự khác biệt giữa u và v:

u là giá trị Pacman kì vọng

v là giá trị thực tế Casper đạt được

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

> Nhánh chứa lá (0,0): Sau khi biết nhánh (6,6) trả về v = 6 cho (0,6), Pacman tại đó đã chọn u=6. Các nhánh nút lá phía bên phải nút (3,9) và (4,8).

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Khi biết Casper thân thiện, trò chơi trở thành trò chơi hợp tác hoàn toàn (MAX-MAX)
Giá trị toàn bộ cây ở gốc thay đổi từ (4, 8) thành 9 (đi theo nhánh phải đến lá (9, 9))

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Yếu tố rẽ nhánh b hữu hạn (b < vô cực).	Chi phí mọi bước đi là như nhau (không âm, bằng nhau $c=c_0$)

Depth-First Search (DFS)	- Cây: Chiều sâu hữu hạn m < vô cực và không có vòng lặp. - Đồ thị: Cần kiểm tra trạng thái đã thăm.	Không tối ưu trong mọi trường hợp.
Iterative Deepening Search (IDS)	Yếu tố rẽ nhánh b hữu hạn (b < vô cực).	Chi phí mọi bước đi là như nhau
A* Search	Yếu tố rẽ nhánh b < vô cực và chi phí bước đi epsilon > 0.	- Đồ thị: Heuristic nhất quán

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

Chi phí bước: Nếu tồn tại chi phí bước đi <= 0 hoặc bằng 0, thuật toán có thể rơi vào vòng lặp vô hạn. Nếu chi phí không đồng nhất, BFS/IDS chỉ tìm thấy lời giải có độ sâu nhỏ nhất chứ không phải đường đi có tổng chi phí nhỏ nhất .

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Tác tử phù hợp: B, C
Giải thích: Môi trường quan sát đầy đủ nên percept hiện tại chứa đủ thông tin về trạng thái thế giới. Tác tử B và C đều đủ khả năng đưa ra hành động tối ưu. A không chọn được vì hành động độc lập hoàn toàn với percept.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Tác tử phù hợp: C
Giải thích: Khi chỉ quan sát một phần, percept hiện tại không đủ để xác định trạng thái thực tế. Tác tử cần tích lũy toàn bộ chuỗi lịch sử cảm nhận percepts để suy luận ra trạng thái hiện tại. B thất bại vì dễ rơi vào vòng lặp vô hạn.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Tác tử phù hợp: C
Giải thích: môi trường quan sát một phần bắt buộc phải dùng chuỗi lịch sử percepts để cập nhật phân phối xác suất/trạng thái niềm tin nhằm chọn hành động có lợi ích kỳ vọng tối đa.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng. Ví dụ: Môi trường mà mọi hành động của tác tử đều đem lại phần thưởng như nhau (ví dụ: môi trường không có mục tiêu, hoặc mọi hành động đều luôn dẫn đến chiến thắng). Khi đó, bất kể tác tử chọn hành động ngẫu nhiên hay cố định (như tác tử A, B, C) thì hành động đó vẫn luôn là hành động hợp lý tối ưu.

— Hết —